

Hotel Management System

1. Requirements Phase (Functional + Security)

In this phase, the project team defines what the hotel management system must do and how it should protect data and services.

1.1 Functional Requirements

Typical functional requirements for a hotel management system include:

- User Management
 - Admin, manager, receptionist, housekeeping staff, and guest roles.
 - Create, update, deactivate staff accounts.
- Room and Inventory Management
 - Add and categorize rooms (standard, deluxe, suite).
 - Track room status (available, occupied, cleaning, under maintenance).
- Reservation and Booking
 - Create, modify, and cancel bookings.
 - Check room availability by date, room type, and occupancy.
- Check-in / Check-out
 - Record guest check-in and assign room.
 - Generate invoices at check-out.
- Billing and Payments
 - Generate bills including room charges, taxes, and additional services.
 - Integrate with secure payment gateways (for online bookings).
- Reports and Analytics
 - Occupancy reports, revenue reports, staff performance summaries.
- Web Interface
 - Responsive web UI for front desk and managers.
 - Optional guest-facing modules for online reservation.

1.2 Security Requirements

Security requirements define how the system will protect confidentiality, integrity, and availability of data:

- Authentication
 - Unique username/email with strong password policy.
 - Password complexity rules (length, character types).
 - Account lockout after several failed login attempts.
- Authorization and Access Control
 - Role-Based Access Control (RBAC), ensuring:
 - Admin can manage all system data and user accounts.
 - Receptionist can manage bookings and check-in/check-out but not system configuration.
 - Housekeeping can only view room cleaning schedules and status.
 - Guests can only access their own booking details.
- Data Protection
 - Sensitive data (passwords, tokens) stored with strong hashing algorithms.
 - Use of TLS/HTTPS for all web traffic to prevent eavesdropping.
- Input Validation
 - Validation for all fields in forms (dates, numbers, text).
 - Restrictions on special characters to reduce injection risks.
- Audit and Logging
 - Logging of critical events (login, logout, failed logins, booking changes, role changes).
 - Secure storage of logs for later security review.
- Compliance and Privacy
 - Minimal collection of personal data.
 - Clear policies for data retention and deletion.

2. Analysis and Risk Assessment Phase

In this phase, requirements are refined and security risks are analyzed.

2.1 Requirements Analysis

- Use-Case Identification
 - Define detailed use-cases such as “Create Booking”, “Cancel Booking”, “Generate Report”, “Update Room Status”.
 - Identify actors (Admin, Manager, Receptionist, Housekeeping, Guest) and their interactions with the system.
- Data Flow Analysis
 - Draw data flow diagrams (DFDs) showing movement of:
 - Guest information from UI to server and database.
 - Booking requests to the database and back.
 - Payment information via payment gateway.
 - Identify trust boundaries (e.g., between browser and server, server and payment provider).

2.2 Threat Modeling and Risk Identification

- Threat Identification
 - For each use-case and data flow, identify potential threats, such as:
 - SQL Injection via booking forms.
 - Cross-Site Scripting (XSS) via user inputs displayed on pages.
 - Broken Access Control on admin or manager pages.
 - Session hijacking and fixation.
- Risk Assessment
 - Evaluate each threat by:
 - Likelihood (low/medium/high).
 - Impact (low/medium/high).
 - Prioritize risks that have high impact and medium or high likelihood.
- Mitigation Planning
 - Mapping threats to controls, for example:
 - SQL Injection → use prepared statements/ORM and strict input validation.

- XSS → output encoding, strict content security policy, validation.
- Broken Access Control → proper RBAC checks on every protected endpoint.
- Session issues → secure session cookies, timeouts, logout mechanism.

3. Secure Design Phase

This phase defines the architecture and design patterns that inherently support security.

3.1 System Architecture Design

- Layered Architecture
 - Presentation Layer (Web UI): HTML/CSS/JS, form validations, user interface.
 - Business Logic Layer: application logic for bookings, billing, roles.
 - Data Access Layer: database operations, ORM, repository classes.
- Separation of Concerns
 - Separate modules for authentication/authorization, booking management, reporting, and notifications.
- Network and Deployment Design
 - Web server separated from database server.
 - Database not directly exposed to the public network.

3.2 Security Design Decisions

- Authentication and Session Management
 - Use secure frameworks' built-in authentication mechanisms where possible.
 - Store session identifiers in HTTP-only, secure cookies.
 - Implement session timeout and explicit logout.
- Authorization
 - Implement centralized RBAC checks:
 - Every protected route or controller checks user role and permissions.
 - No direct access to admin or manager functions by URL guessing.
- Data Protection and Privacy by Design
 - Hash passwords with strong algorithms (e.g., bcrypt/argon2) and salts.

- Encrypt highly sensitive data if stored (e.g., tokens, certain identifiers).
- Avoid storing full payment card details; rely on payment gateway tokens.
- Secure Error Handling
 - Show generic error messages to end users.
 - Log detailed errors on the server for debugging.
- Defense in Depth
 - Input validation on client and server.
 - Output encoding in views.
 - Use of security headers such as:
 - Strict-Transport-Security (HSTS).
 - X-Content-Type-Options.
 - X-Frame-Options or frame-ancestors via CSP.

4. Secure Implementation Phase

This phase converts the design into code using secure coding practices.

4.1 Coding Standards

- Adopt a secure coding guideline (such as OWASP recommendations).
- Follow consistent naming conventions, code comments, and modularization for maintainability.
- Avoid copying untrusted code snippets without review.

4.2 Secure Coding Practices

- Input Validation
 - Use server-side validation as the final gate, even if client-side validation exists.
 - Validate format, length, and type for each field (e.g., dates, numbers, emails).
- Output Encoding
 - Encode user-supplied data before rendering in HTML, attributes, or scripts.
- Safe Database Access
 - Use parameterized queries or ORM to prevent SQL injection.

- Restrict database user permissions to only necessary operations.
- Authentication and Password Handling
 - Never store passwords in plain text.
 - Use strong password hashing and compare hashes on login.
- Session Handling
 - Regenerate session IDs after login.
 - Use secure, HTTP-only cookies; avoid putting session IDs in URLs.
- Error and Exception Handling
 - Handle exceptions gracefully; log exceptions with appropriate context.
 - Ensure error messages do not reveal stack traces or database details to users.
- Secure Use of Third-Party Libraries
 - Use well-maintained libraries and frameworks.
 - Regularly update dependencies to patch known vulnerabilities.

5. Security Testing Phase

Testing ensures both functionality and security controls work as intended.

5.1 Functional Testing

- Unit Testing
 - Test individual functions such as creating a booking, calculating bill, or changing room status.
- Integration Testing
 - Test interactions between modules, such as booking with billing and notification.
- System Testing
 - Validate that the system meets all specified functional requirements.

5.2 Security-Focused Testing

- Input Validation Testing
 - Try unexpected input: long strings, special characters, scripts tags, SQL-like strings.

- Authentication and Authorization Testing
 - Attempt to access admin or manager URLs with a low-privilege account.
 - Test session timeout, logout, and multiple login scenarios.
- Vulnerability Scanning (if possible)
 - Use automated tools (where available) to scan for common vulnerabilities such as:
 - SQL injection.
 - XSS.
 - Insecure cookies.
- Penetration Testing (Basic)
 - Manually test:
 - Bypassing client-side validation.
 - Directly calling endpoints via tools like Postman.
- Regression Testing
 - After fixing vulnerabilities, re-run tests to ensure no new issues were introduced.

6. Deployment and Environment Hardening Phase

This phase focuses on safely deploying the application and securing its environment.

6.1 Deployment Preparation

- Configuration Management
 - Use separate configuration files for development and production.
 - Ensure production uses strong credentials and secure settings.
- Secrets Management
 - Do not hard-code passwords or API keys in source code.
 - Store secrets in environment variables or secure vaults.

6.2 Server and Network Hardening

- Operating System and Server Security
 - Apply latest security patches to OS, web server, and database.

- Disable unused services and ports.
- Web Server Configuration
 - Enforce HTTPS using a valid SSL/TLS certificate.
 - Configure security headers (HSTS, Content Security Policy, etc.).
- Database Security
 - Use strong passwords for database accounts.
 - Restrict remote access; allow only necessary hosts to connect.
- Logging and Monitoring Setup
 - Enable application and server logs.
 - Configure log rotation and secure storage of logs.

7. Maintenance, Monitoring, and Continuous Improvement Phase

After deployment, the system must be maintained securely over time.

7.1 Ongoing Maintenance

- Patch Management
 - Regularly update frameworks, libraries, and server software.
- Backup and Recovery
 - Schedule regular backups for databases and important configuration files.
 - Periodically test backup restoration procedures.

7.2 Monitoring and Incident Handling

- Monitoring
 - Continuously monitor logs for:
 - Suspicious login patterns.
 - Repeated failed login attempts.
 - Unusual access to admin features.
- Incident Response (Basic Plan)
 - Define steps to handle security incidents:
 - Identify and contain the issue.

- Analyze logs and impact.
- Apply fixes and patches.
- Notify relevant stakeholders if required.

7.3 Continuous Improvement

- Periodic Security Review
 - Schedule regular audits of roles, permissions, and logs.
 - Review whether new features introduce additional risks.
- User Feedback
 - Collect feedback from staff and managers about usability and potential security issues (e.g., confusing error messages, unclear login flows).
- Updating Documentation
 - Keep technical and user documentation updated when new security controls or features are introduced.